

GS_FLOATERLAB · EXP52 → EXP56

Incremental 3DGS Mapping 실시간화

VIGS-SLAM 채택 리캡 · 실시간 최초 돌파 · 품질까지 함께 개선
"왜 안 빨라지나" 심층 규명

1253 시퀀스 · exp52~56 · 07/20 → 07/27

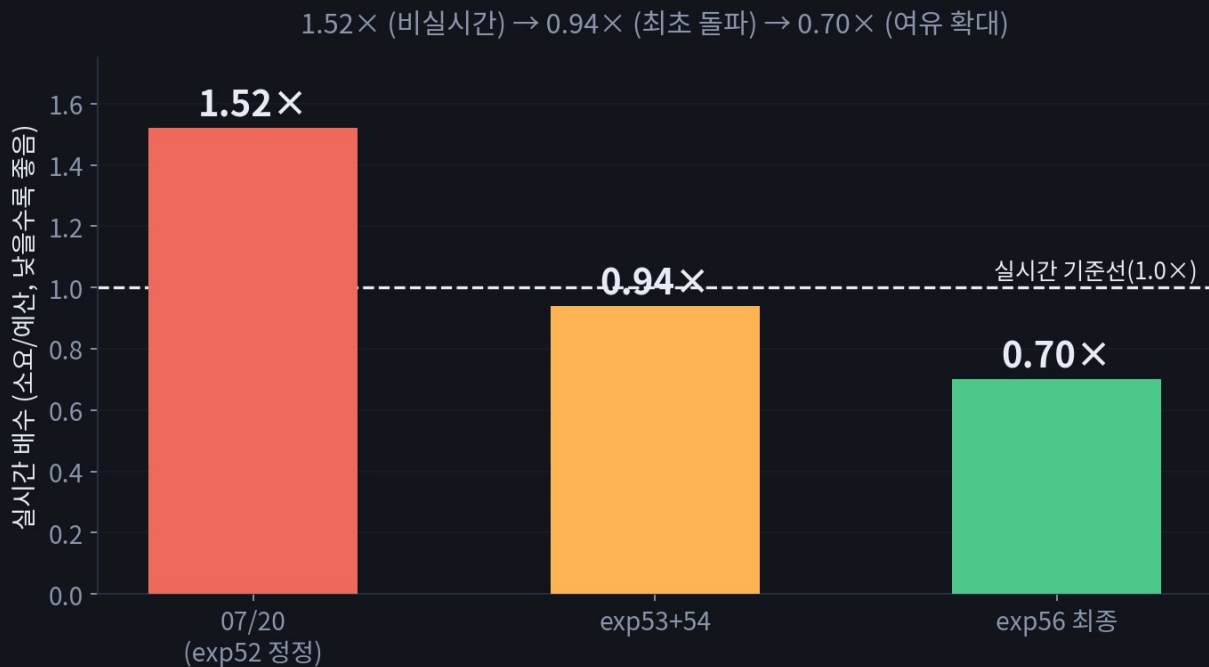
지난 발표 리캡: 어디까지 끝냈나

- VIGS-SLAM(ECCV2026) 채택 — 단안 RGB+IMU, DROID-SLAM식 dense correlation 트래킹. 우리 목표(흑백 SLAM 트래킹 + RGB incremental 매핑)에 가장 가까운 참조 아키텍처
- 20초 타이밍 버그 발견·수정 — reader 프로세스 sleep(20) 순서 문제로 모든 "온라인 루프 총합"에 인위적 20초가 섞여있었음. 수정 후 실시간 배수 2.77배 → 1.52배로 정정
- 구조적 개선 — IMU 프리적분 C++ 빌드 + TensorRT 3종(-14%), tracking/mapping 비동기 오버랩 gs_parallel 도입(업스트림 레이스 컨디션 버그 발견·수정, -26.1% 추가)
- 핵심 진단 — mapping(rasterize+backward+loss_compute)이 온라인 루프의 81.4%로 압도적 병목. 이번 사이클(exp53~56) 전체가 이 진단 위에서 진행됨

오늘은 이 지점(1.52배, 아직 실시간 아님) 이후 일주일간 일어난 일만 다룹니다.

SUMMARY

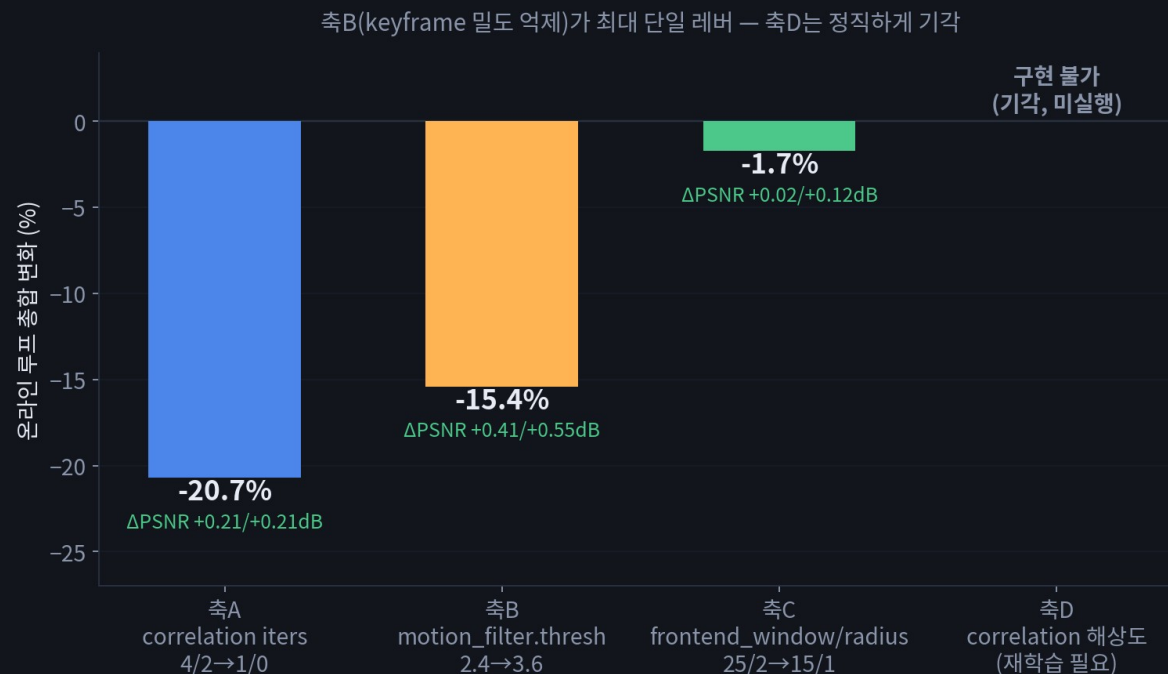
이번 사이클 요약 (TL;DR)



- exp53+54: 트래킹·매핑 양쪽 경량화 → 실시간 최초 돌파(0.94배)
- exp55: 내용-적응 gaussian 예산 + carve loss 온라인 이식 → 시간 유지하며 평균 gaussian -35.9%, 가시 floater -7.5%
- exp56: "왜 안 빨라지나"를 회귀분석으로 규명 → 숨어있던 진짜 병목 2건 발견·수정 → 0.70배, PSNR까지 +0.88/+0.93dB 개선

1.52배(비실시간) → 0.70배(여유 19초), 그 사이 품질은 계속 좋아짐

프론트엔드(트래킹) 경량화



- 07/20 진단(mapping 81.4%)과 별개로 트래킹도 손덜 여지가 컸음 — 나중 exp56 부록에서 "순수 트래킹 절감폭(-47.5%)이 매핑 절감폭(-12~15%)보다 훨씬 크다"는 게 재확인됨
- 축D(correlation 해상도)는 사전학습 가중치에 shape가 고정 결합돼 조사 후 구현 불가 판정 — 재학습 없인 못 건드릴, 정직하게 기각 기록

재학습 없이 안 되는 것도 시도해보고 명확히 접었다는 게 중요 — 막연히 미룬 게 아니라 판정을 끝냄.

트래킹 측A~D, 실제로 뭘 바꾸는 건가

파이프라인: 카메라 프레임 → motion_filter(이게 keyframe인가?) → frontend(correlation → GRU 반복정제 → local BA)

측A

correlation iters — 두 프레임의 optical flow(correlation)를 GRU로 여러 번 반복해서 다듬는 횟수(iters1/iters2). 시간 -20.7%, Δ PSNR +0.21/+0.21dB(오히려 개선) — 4/2회를 1/0회로 줄여도 정확도 손실 없었음

측B

motion_filter.thresh — 새 프레임이 이전 keyframe 대비 "이만큼은 움직여야 keyframe으로 인정"하는 임계값. 시간 -15.4%, Δ PSNR +0.41/+0.55dB — keyframe 자체가 덜 생겨 트래킹·매핑 작업량이 동시에 줄어들, 이 세션 최대 레버

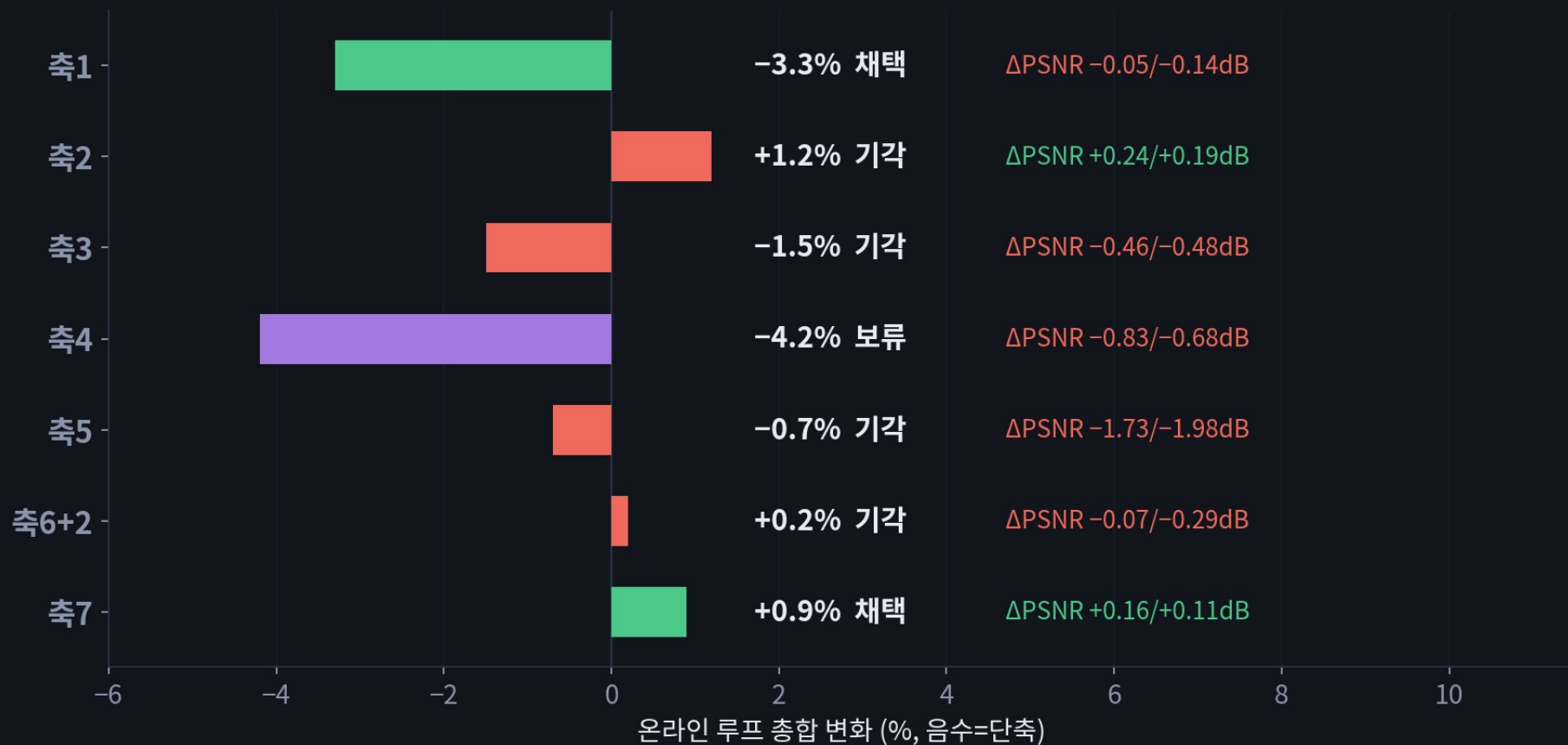
측C

frontend_window/radius — local BA가 한 번에 붙잡고 최적화하는 "최근 keyframe 그래프"의 크기(window)와 이웃 범위(radius). 시간 -1.7%, Δ PSNR +0.02/+0.12dB — 줄이면 BA solve 행렬이 작아져 계산이 가벼워짐

측D

correlation 해상도 — correlation volume(두 프레임 유사도 맵)을 뽑는 특징맵의 해상도. 사전학습된 신경망 가중치의 입력 shape에 못박혀 있어 재학습 없인 못 건드릴 — 미실행

매핑 경량화 — 7축 스캔



속도 축만 스캔한 게 아니라 축3에서 우연히 "이 시점엔 트래킹이 91%"라는 걸 발견 — 이게 exp53 우선순위의 직접 근거가 됨(서로 다른 실험이 서로의 근거를 만든 사례). 각 축이 정확히 뭘 바꾸는 실험인지는 다음 슬라이드에서 설명.

매핑 축1~7, 실제로 뭘 바꾸는 건가

파이프라인: keyframe 도착 → 초기 gaussian 샘플링(축1/2/7) → map() 반복 최적화(축3) → rasterize · densify(축4/5/6)

축1

pcd_downsample — 매 keyframe마다 RGB-D에서 새 gaussian을 뽑을 때의 샘플링 간격. 64→128은 점 개수 절반. 시간 -3.3%, ΔPSNR -0.05/-0.14dB(거의 동일) — 채택

축2

pcd_downsample_init — 축1과 같은 개념인데 "시퀀스 맨 첫 keyframe"에만 적용. 시간 +1.2%(역효과), ΔPSNR +0.24/+0.19dB — densify가 나중에 보충해버려 시간 이득이 상쇄됨, 기각

축3

map() iters — keyframe 하나를 처리할 때 gaussian을 SGD로 최적화하는 반복 횟수. 시간 -1.5%, ΔPSNR -0.46/-0.48dB — 이 시점엔 트래킹이 병목(91%)이라 ROI 나뭇, 기각

축4

render_downsample — 매핑용 렌더링 자체를 낮은 해상도로 그림(원래는 트래킹 해상도의 8배까지 업샘플). 시간 -4.2%, ΔPSNR -0.83/-0.68dB — 유효하나 이미 실시간이라 손실 감수 안 함, 보류

축5

max_viewpoints — map() 한 번의 호출에서 동시에 보는 카메라 뷰 개수의 상한. 시간 -0.7%(거의 무변화), ΔPSNR -1.73/-1.98dB — 최악의 ROI, 기각

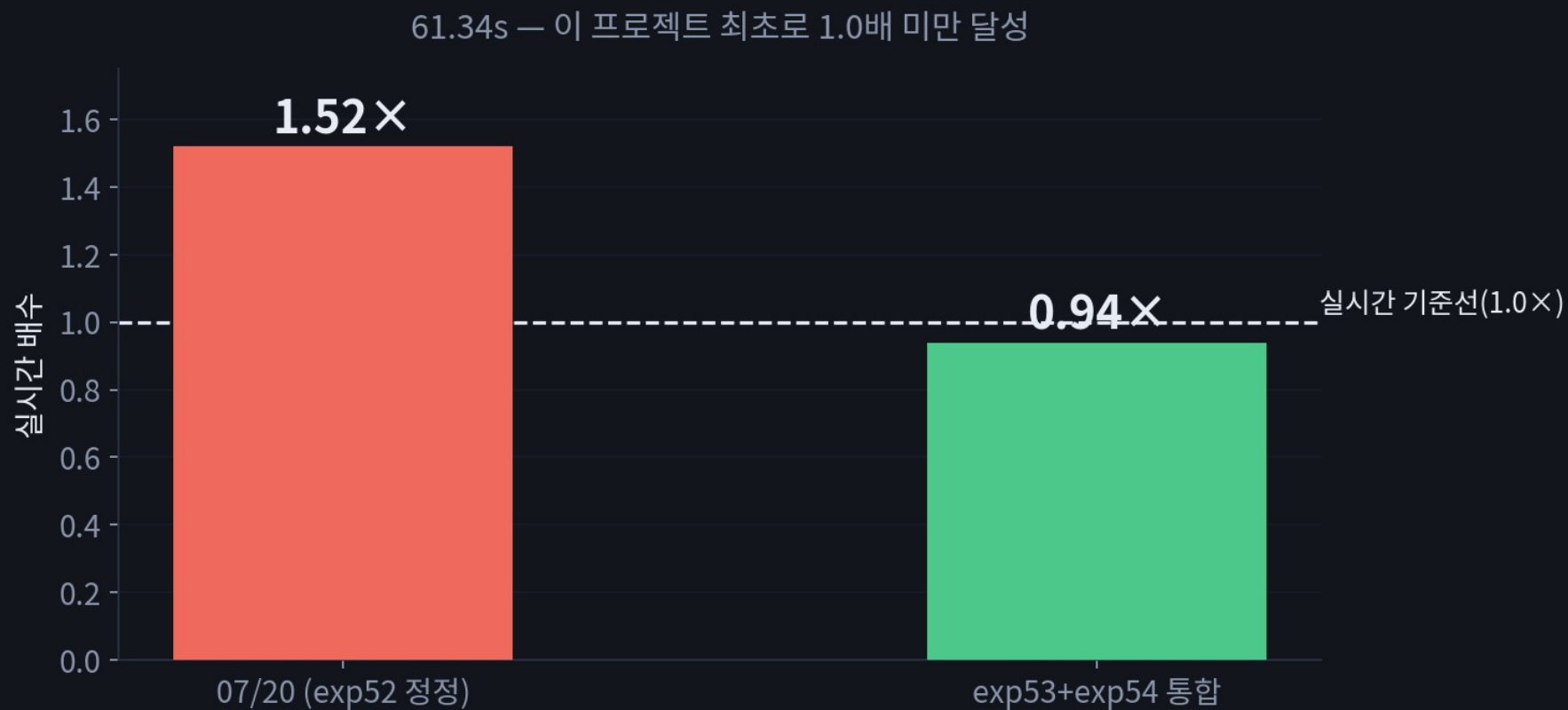
축6

densify_grad_threshold — gaussian이 "부족하다"고 판단해 새로 쪼개거나 복제하는(densify) 공격성을 정하는 임계값. 시간 +0.2%, ΔPSNR -0.07/-0.29dB — 개수는 줄여도 시간은 그대로, 기각

축7

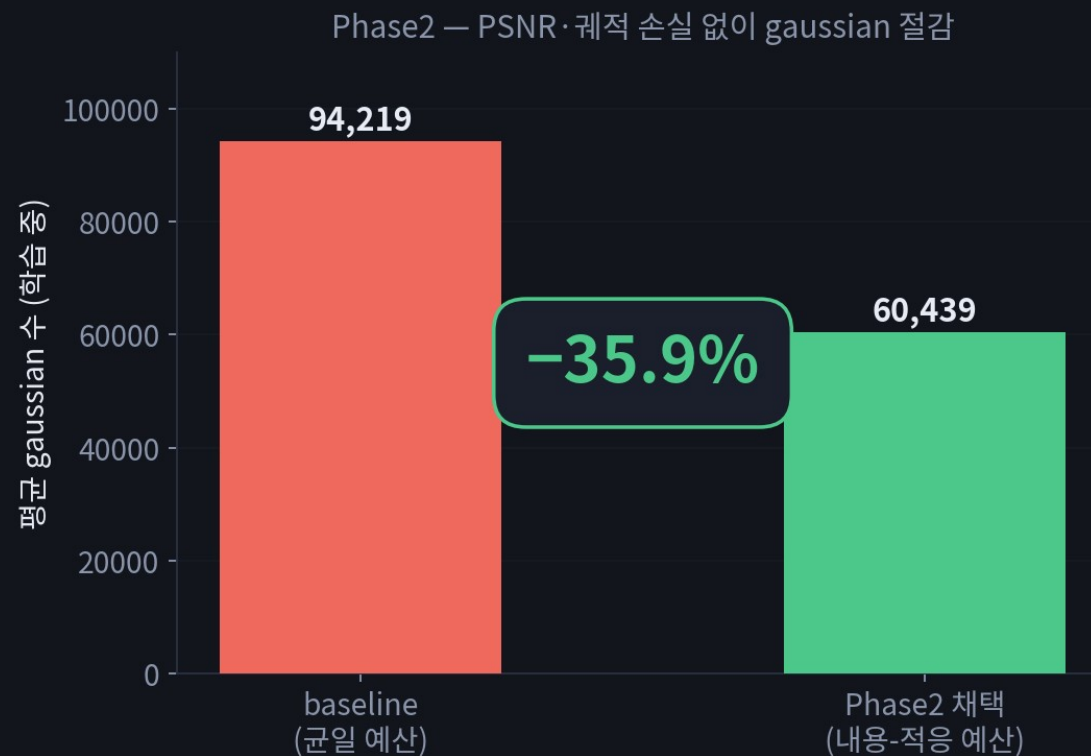
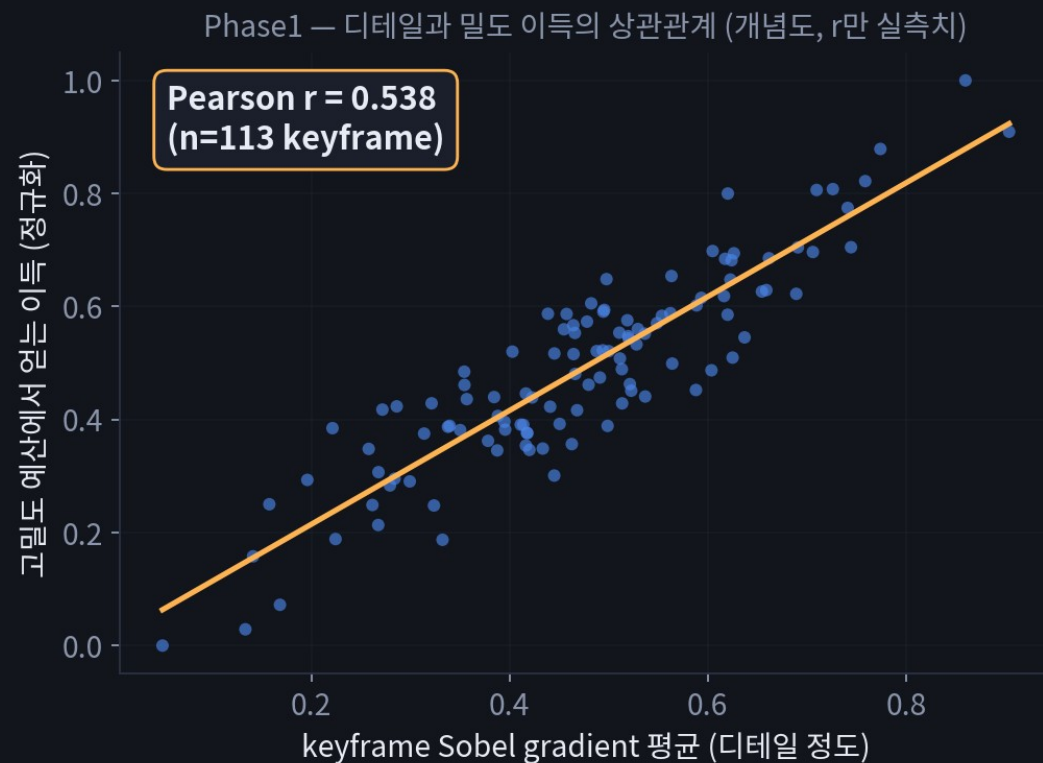
PPM 적응 샘플링 — 균일 샘플링 대신 이미지 디테일(Sobel gradient)에 따라 점을 더/덜 배분. 시간 +0.9%(오차범위), ΔPSNR +0.16/+0.11dB — 같은 예산에서 품질만 공짜로 오름, 채택

결과: 실시간 최초 돌파



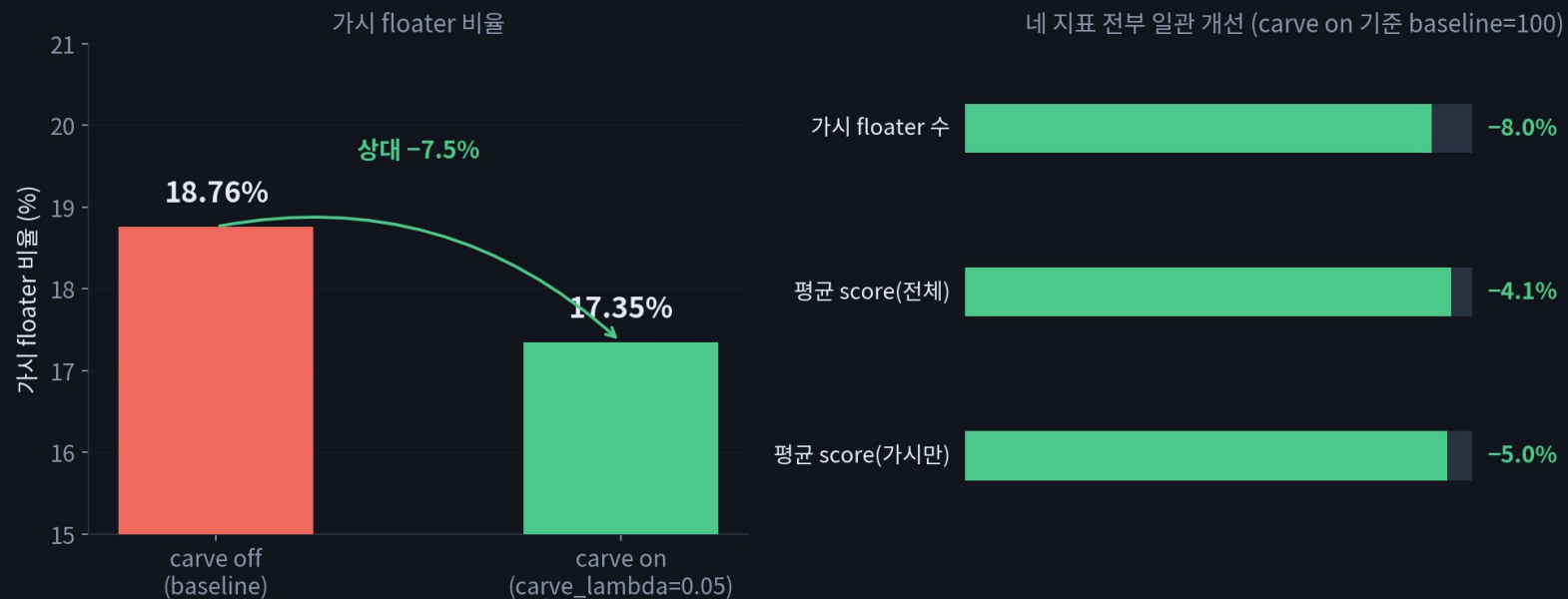
exp53+exp54 통합 레시피: 61.34초, 실시간 배수 0.94배 — 이 프로젝트 최초로 1.0배 미만 달성. 여기서 멈춰도 목표(실시간)는 달성인데, 다음 두 사이클(exp55/56)에서 여유를 더 벌고 품질까지 올림.

내용-적응 gaussian 예산



가설: 디테일 많은(Sobel gradient 큰) keyframe엔 gaussian이 더 필요하고, 단조로운 keyframe엔 덜 필요할 것. Phase1(캘리브레이션)에서 상관관계로 먼저 검증한 뒤 Phase2에서 배율곡선+keyframe별 cap을 구현 → 평균 gaussian 수 -35.9%, 최종 -35.3%, PSNR·궤적 손실 없음(오히려 소폭 개선). 사용자 아이디어를 감으로 가지 않고 상관관계로 먼저 검증한 뒤 구현했다는 게 핵심.

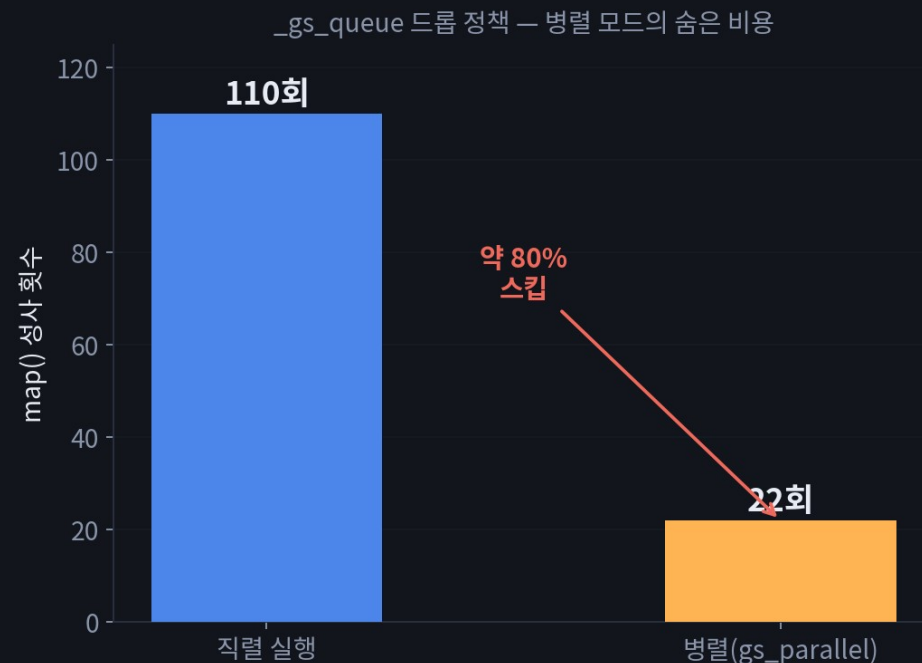
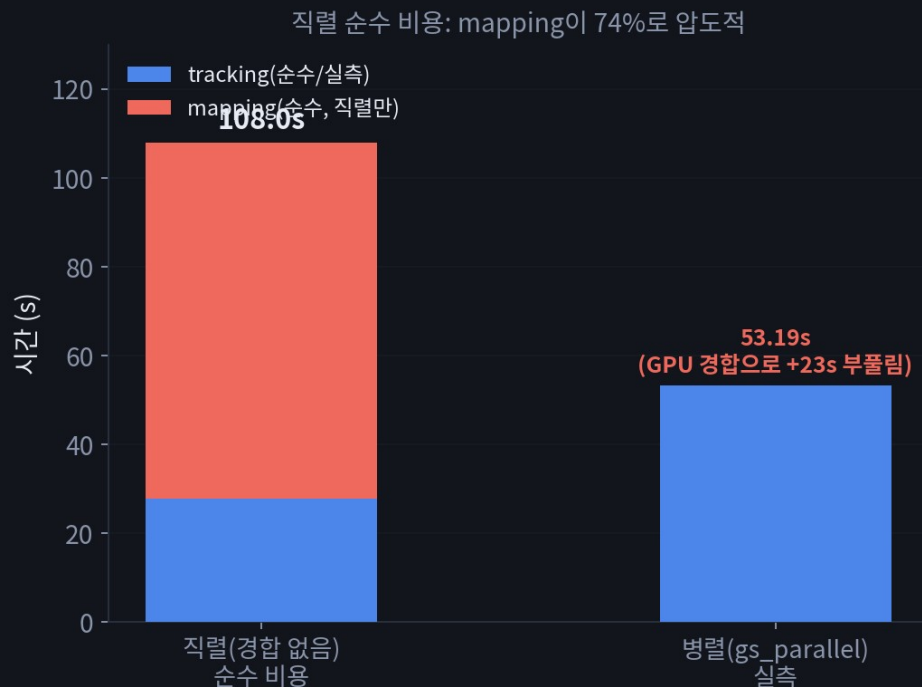
carve loss 온라인 이식



- 배치 트랙에서 이미 검증된(AUC 0.98) carve loss를, VIGS의 BA-정제 추적 depth(disps_up)를 신뢰 표면 삼아 depth-violation 전용 온라인 근사로 재설계
- region GT가 VIGS 좌표계에 안 맞아 — carve_loss.py 자신의 신호 설계를 그대로 재구현한 오프라인 진단 지표를 새로 제작해 검증

표준 지표가 안 맞으면 새로 만들어서라도 채택/기각을 숫자로 판단한다는 원칙.

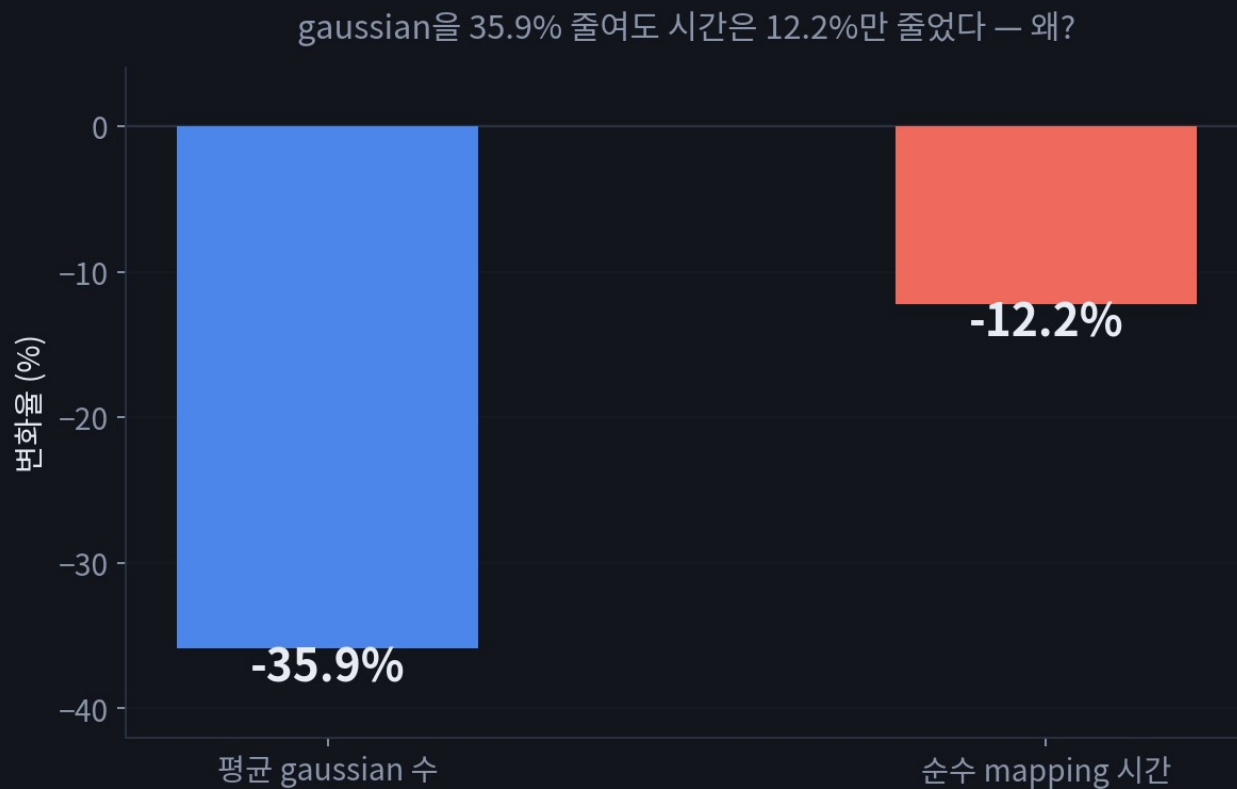
부록: 직렬 실행으로 밝혀진 진짜 구조



- "지금 상태로 직렬 돌리면 순수 시간이 어떻게 되나" 실측 → 순수 tracking 27.9초 vs 순수 mapping 80.1초. 이게 exp54의 "tracking-bound" 결론과 정면 모순
- 파고든 결과 두 가지 발견: ① GPU 경합이 병렬 tracking을 거의 2배로 부풀림 ② _gs_queue가 가득 차면 오래된 keyframe을 버리는 정책 때문에 병렬 모드가 mapping 업데이트의 약 80%를 그냥 스킵(직렬 110회 vs 병렬 22회)

이 발견이 exp56 전체의 출발점 — "병렬 배수만 보면 안 되고 안에서 뭐가 스킵되는지 봐야 한다".

문제 제기: gaussian ↓ 인데 시간은 그대로

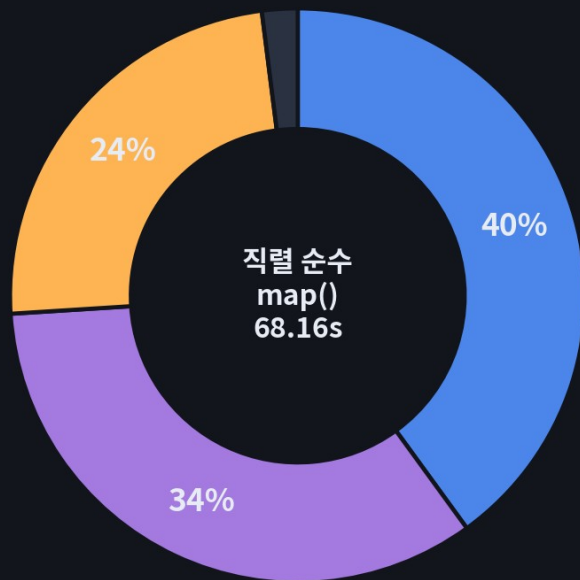


- exp55에서 평균 gaussian 수를 35.9% 줄였는데 순수 mapping 시간은 12.2%밖에 안 줄었음 — "gaussian 수를 줄이는" 접근이 한계에 도달한 신호
- exp54 축6+2에서도 이미 같은 현상 관측(gaussian 수를 더 늘려도 시간 그대로)

질문: gaussian 수가 시간을 안 줄이면, 무엇이 시간을 지배하는가?

기존 계측 재분석으로 원인 진단

전부 $\text{iters} \times n_{\text{view}}$ 에 곱으로 걸림 — 고정비가 지배적

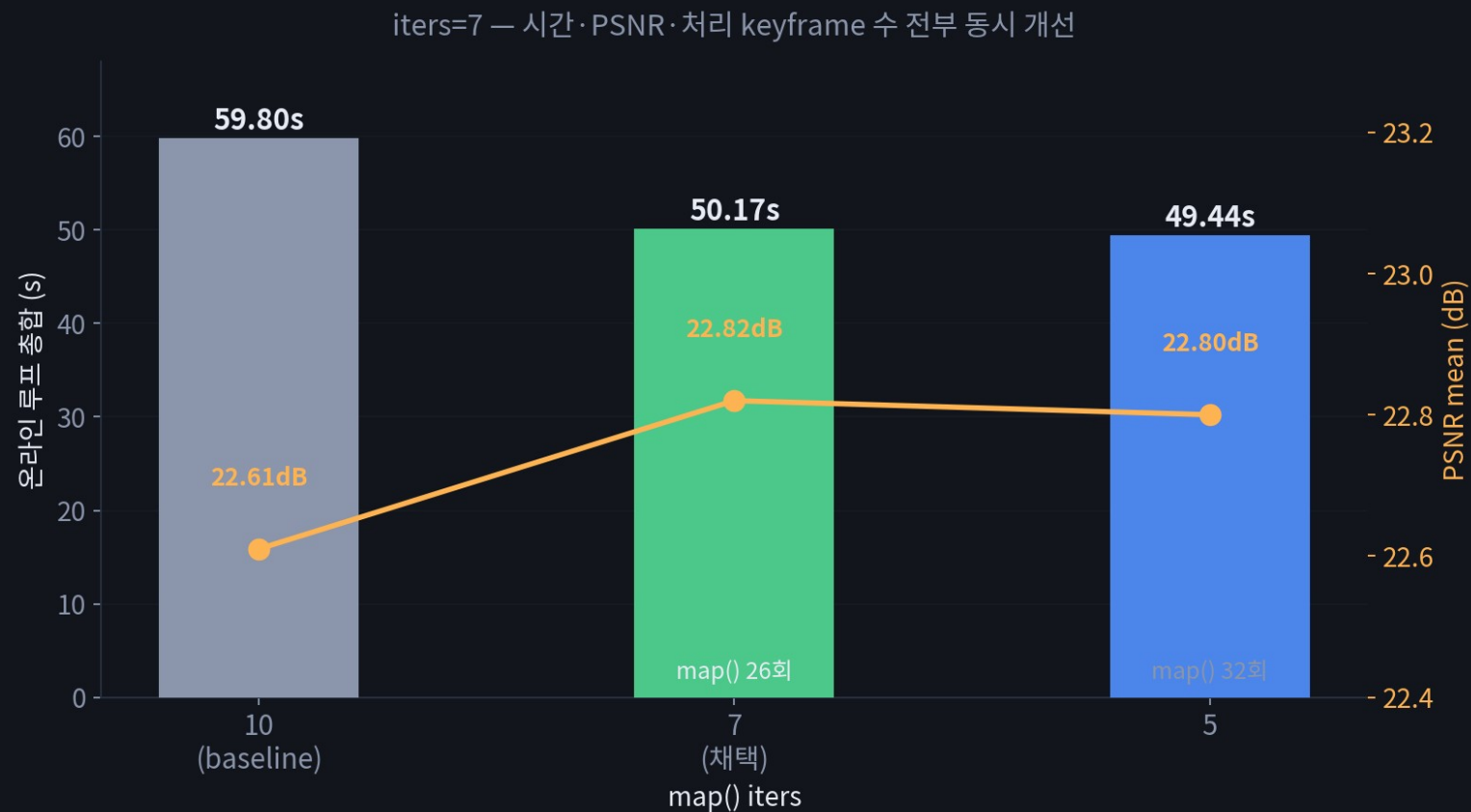


- rasterize
(N+픽셀 혼합)
- backward
(N+픽셀 혼합)
- loss_compute
(순수 픽셀, N무관)

- 새 실험 없이 기존 타이밍 로그를 처음으로 세부 태그별 집계 — 직렬 순수 map() 68.16초 중 rasterize 40% + backward 34% + loss_compute 24%
- loss_compute는 순수 픽셀 연산(N-무관), rasterize/backward도 이 gaussian 규모(85k~130k)에선 "고정비"가 데이터량-비례 항을 압도함을 확인

새 실험을 돌리기 전에 이미 있는 데이터부터 다시 봤다 — 이게 다음 발견들의 방법론.

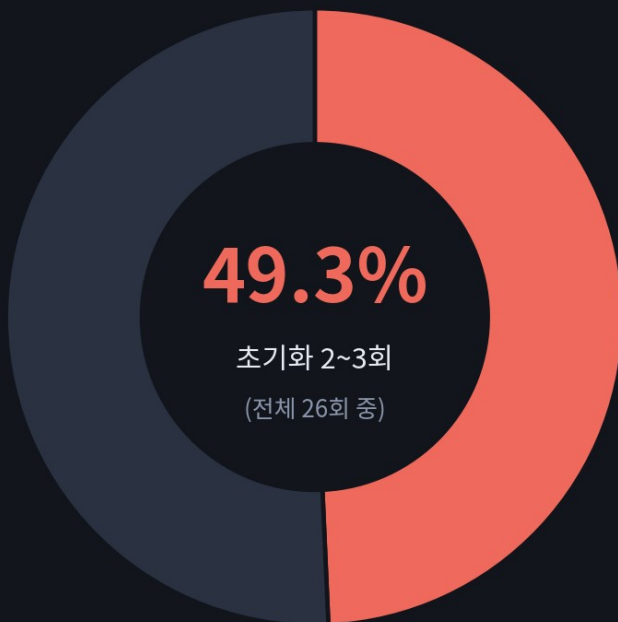
iters 하향 — 첫 큰 개선



"고정비가 지배적"이라면 진짜 변수는 gaussian 수가 아니라 "반복 횟수(iters)" — keyframe당 SGD 반복 10→7→5 스캔. iters=7: 시간 -16.1%, PSNR mean/kf 둘 다 +0.21dB, map() 처리 keyframe 수도 22→26회 증가 — 전 지표 동시 개선. (참고) 반대 방향(iters를 올려보는 실험)은 오히려 coverage가 줄어 역효과.

숨어있던 진짜 병목 발견

map() 호출 26회 중 단 2~3회가 mapping 시간의 절반



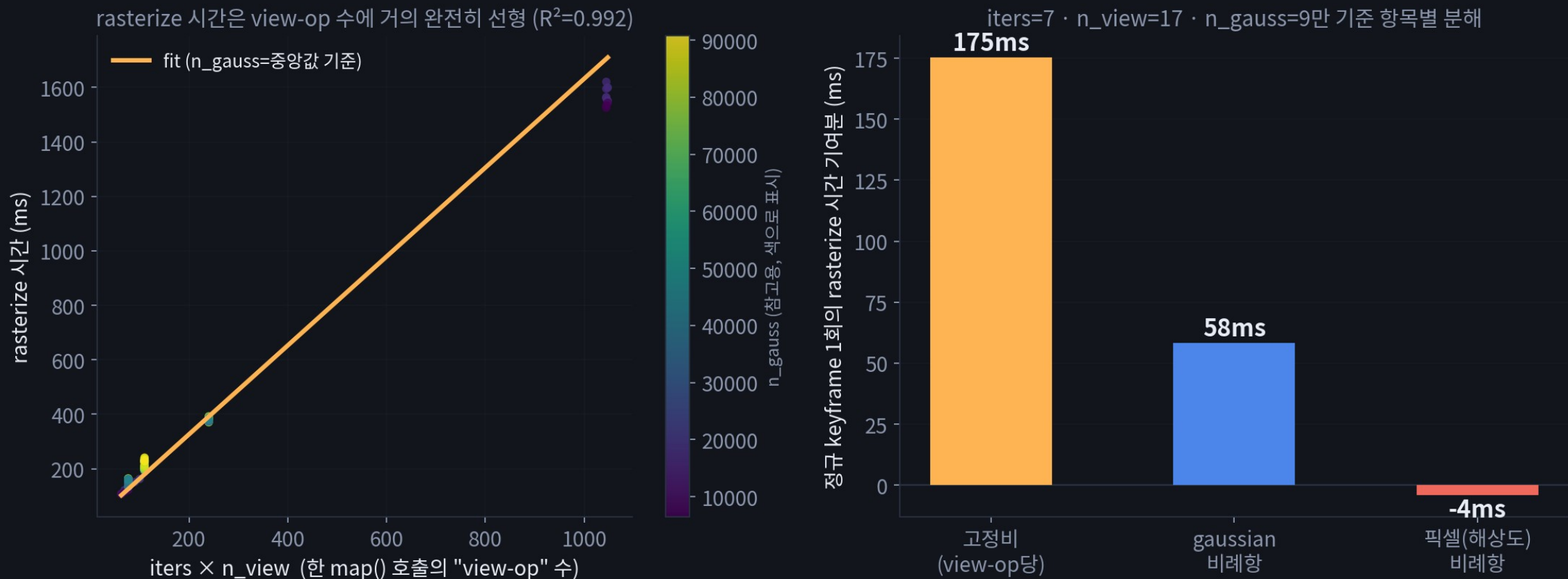
- 초기화/재초기화 2~3회
(iters=90~131)
- 일반 keyframe 21회
(iters=7)

- map() 호출을 세부 로그로 종류별 분해해보니 — 26회 중 단 2~3회(맵 최초 초기화 + IMU 재초기화)가 전체 mapping 시간의 49%를 차지
- 원인: IMU 재초기화 시 맵 전체를 삭제하고 처음부터 다시 채우는데, 이때 반복 횟수가 90~131회(정규 호출의 10배 이상)
- init_itr_num 1050→600으로 낮춰 추가 시간 -6.2%, PSNR 사실상 무손실(300까지 내리면 실손실 확인 후 기각)

호출 26개 중 딱 2~3개에 절반이 몰려있었다는 게 이번 사이클 최대 발견.

회귀분석 피팅 — 무엇이 시간을 결정하는가

Phase 5 — 548개 실제 map() 호출 로그를 회귀분석으로 피팅 (직렬, GPU 경합 없음)



왼쪽: 548개 실제 map() 호출을 $iters \times n_view$ (한 호출이 처리하는 "view-op" 수) 하나로 설명하면 $R^2=0.992$ — gaussian 수(점 색깔)는 같은 x값 안에서도 시간을 거의 안 바꿈. 오른쪽: 그 회귀식을 항목별로 분해하면 "view-op당 고정비"가 gaussian-비례항의 3배, 픽셀-비례항은 사실상 0에 가까움.

실제로 피팅된 4개 관계식

직렬(GPU 경합 없음) 330개 실제 map_call 로그, 최소제곱(least-squares) 피팅

iters 그 map() 호출에서 gaussian을 SGD로 최적화하는 반복 횟수 (정규 keyframe=7, 초기화=90~131 등)

n_view 그 반복 1회마다 렌더링해서 loss를 재는 카메라 뷰 개수 (window+global 결눈질 합)

n_gauss 그 시점의 전체 gaussian 개수

pixratio 렌더 해상도 배율 = $1/\text{render_downsample}^2$ (원해상도면 1)

ncalls = $\text{iters} \times \text{n_view}$ — 한 번의 map() 호출이 처리하는 총 "view-op" 수 (rasterize/loss_compute는 view마다 도니까 이 단위가 자연스러움)

rasterize ($R^2 = 0.992$)

$$\begin{aligned} &= 1.473 \cdot \text{ncalls} \\ &+ 0.00545 \cdot \text{ncalls} \cdot (\text{n_gauss}/1000) \\ &- 0.0346 \cdot \text{ncalls} \cdot \text{pixratio} \quad [\text{ms}] \end{aligned}$$

loss_compute ($R^2 = 0.994$)

$$\begin{aligned} &= 0.956 \cdot \text{ncalls} \\ &+ 0.00207 \cdot \text{ncalls} \cdot (\text{n_gauss}/1000) \\ &- 0.0241 \cdot \text{ncalls} \cdot \text{pixratio} \quad [\text{ms}] \end{aligned}$$

backward ($R^2 = 0.929$)

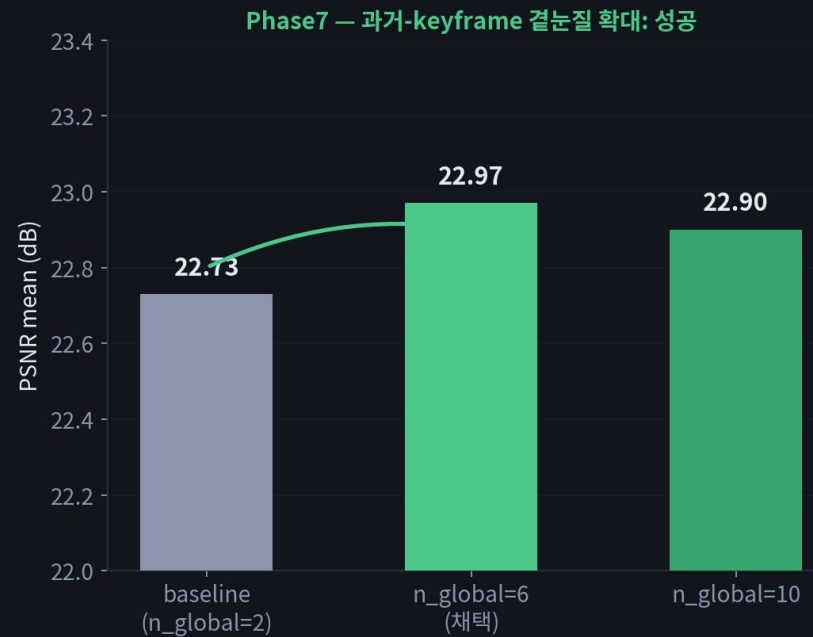
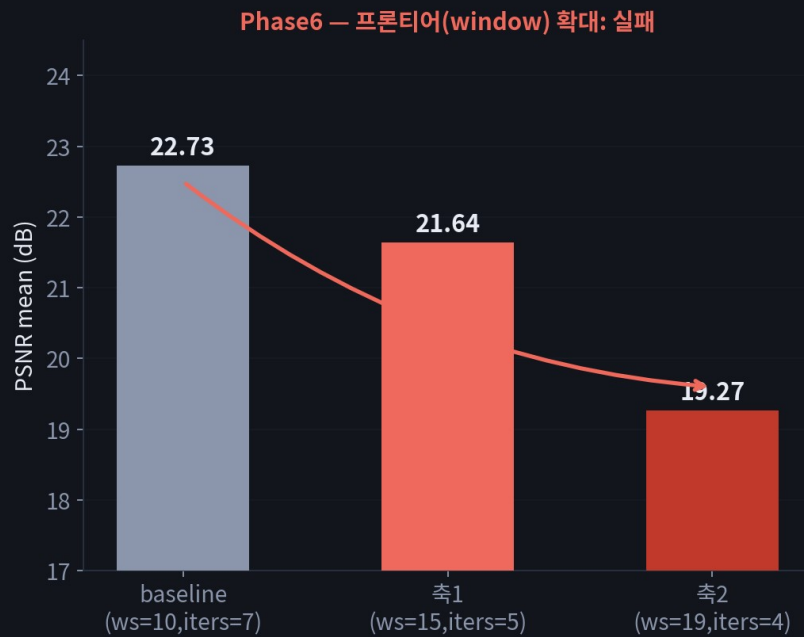
$$\begin{aligned} &= -1.623 \cdot \text{iters} \\ &+ 1.126 \cdot \text{iters} \cdot \text{n_view} \\ &+ 0.126 \cdot \text{iters} \cdot (\text{n_gauss}/1000) \quad [\text{ms}] \end{aligned}$$

optimizer_step ($R^2 = 0.998$)

$$\begin{aligned} &= 1.739 \cdot \text{iters} \\ &- 0.0946 \cdot \text{iters} \cdot \text{n_view} \\ &+ 0.00093 \cdot \text{iters} \cdot (\text{n_gauss}/1000) \quad [\text{ms}] \end{aligned}$$

네 식 전부 " $\text{iters} \cdot \text{n_view}$ " 항의 계수가 gaussian/픽셀 항보다 압도적으로 큼(또는 그 항 자체가 없음) — 그래서 " n_view (카메라 수)가 시간을 지배한다"는 결론이 감이 아니라 계수로 확정됨.

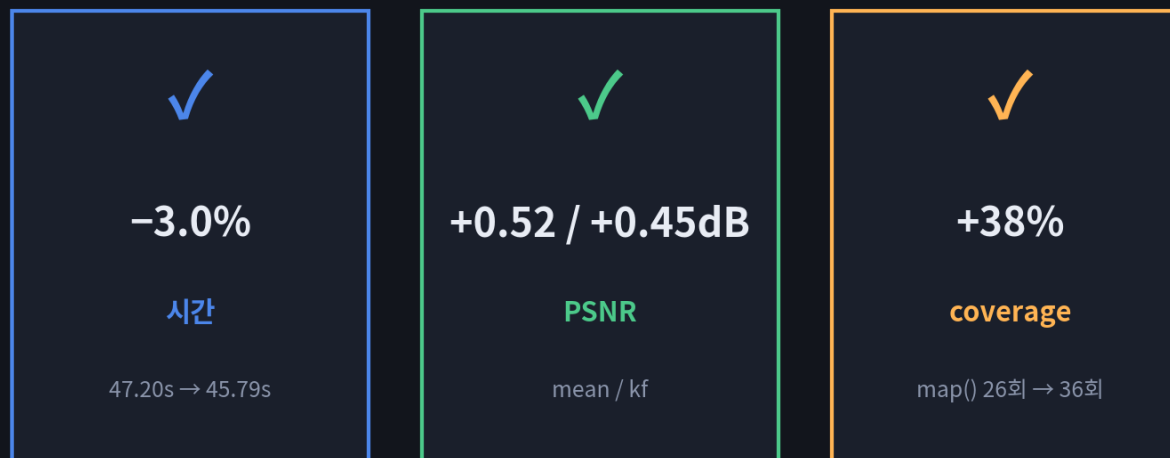
회귀분석으로 규명한 관계식, 그리고 품질까지



- 548개 실제 map() 호출 데이터로 회귀분석 — $\text{iters} \times \text{n_view}$ (반복 횟수 $\times$ 카메라 수)가 시간을 지배, gaussian 수 · 해상도는 부차적($R^2=0.93\sim0.998$)
- "카메라 뷰를 늘리면 품질이 좋아지지 않을까?" → window(프론티어) 자체를 키우면 오히려 PSNR 악화(-1.1~3.5dB) — 기각. 대신 프론티어는 그대로 두고 과거-keyframe 결눈질 비중만 늘리기 → PSNR 개선, 시간 비용 무시할 수준 — 채택

같은 "뷰를 늘린다"는 아이디어인데 방법에 따라 정반대 결과 — 다음 로드맵(dense-frame supervision)에 대한 경고이기도 함.

"공짜" 최적화 발견



그래디언트 수학은 전혀 안 건드림 — camera_utils.py 행렬 캐싱만 추가

- 프로파일링 중 우연히 발견: 카메라 pose가 안 바뀌는데도 매 렌더링 호출마다 행렬 역산을 처음부터 다시 계산하고 있었음
- pose가 바뀌는 지점이 코드 전체에 딱 한 곳뿐임을 확인한 뒤 캐싱 추가 — 그래디언트 수학은 전혀 안 건드리는 무위험 변경

제일 큰 인프라 작업(다음 슬라이드)을 준비하다가 발견한 제일 작은데 제일 확실한 개선.

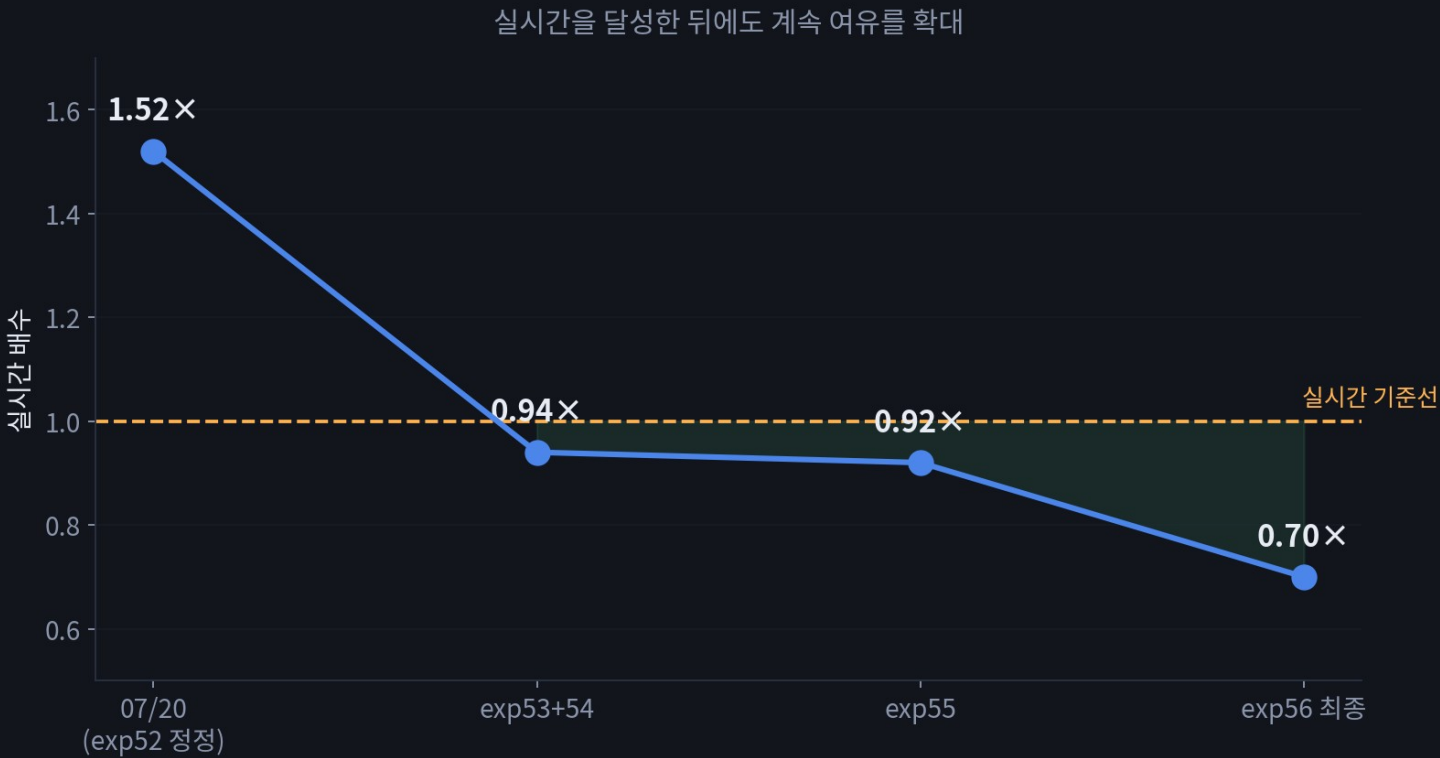
CUDA 커널 batch화 시도 — 정직한 실패 기록



- "카메라 여러 대를 한 번의 CUDA 커널 호출로 묶으면 더 빠르지 않을까" — 실제로 구현·검증까지 진행
- 기존 커널(그래디언트 수학 포함)은 안 건드리고 C++에서 안전하게 감싸는 방식으로 구현, forward는 완전 일치, backward도 수치적으로 검증 통과
- 실전 적용 1차 시도에서 PSNR 붕괴 발견 → 원인(텐서 shape 불일치) 진단·수정 → 재검증 통과
- 하지만 최종 시간 측정 결과 개선 없음 — 진짜 병목이 "CUDA 커널 실행 자체"라는 사전 진단이 재확인됨, 채택하지 않고 안전하게 원복

성공한 것만 보여주면 신뢰가 안 감 — 위험한 시도를 안전하게 검증하고, 안 되면 되돌리는 과정 자체가 방법론.

전체 타임라인



시점	실시간 배수	PSNR(mean /kf)
07/20 (정정 후)	1.52배	—
exp53+54	0.94배	—
exp55	0.92배	22.61/22.95
exp56 최종	0.70배	23.49/23.88

실시간을 달성한 뒤에도 멈추지 않고 추가로 시간을 1/4 줄이면서 품질까지 올렸다.

현재 상태 스코어카드 + 다음 후보

45.79s

온라인 루프 총합
(예산 65.1s 대비 여유 19s)

23.49/23.88

PSNR (mean/kf)

0.70×

실시간 배수

다음 후보 (우선순위 논의 필요)

① 고위험

CUDA 커널 레벨 batch화(Phase 8b가 실패한 진짜 원인 해결) — 이득 불확실

② 저위험

init_itr_num/iters 더 세밀한 스캔 — 이득 작음

③ 신규 후보

오프라인 색정제를 실시간 호환으로 재설계 — 다음 슬라이드, +3~6dB 확인됨

④ 다음 단계

Localization(흑백 SLAM 트래킹) 실제 통합 — exp50과 합류

carve loss로 floater 억제는 유지 중. ①번은 리스크 대비 이득이 불확실해서 팀 논의가 필요한 지점.

부록: 오프라인 후처리가 사주는 PSNR

같은 시점 렌더링 — held-out PSNR mean / keyframe PSNR mean

baseline (07/20)



22.60 / 22.92 dB

exp56 최종 (online만)



23.49 / 23.88 dB

+ 후처리 (26k iter 색정제)



26.53 / 30.33 dB

~196s

후처리 고정비용
(26,000 iter, 프레임수 무관)

+3.04 / +6.45dB

PSNR 이득(mean/kf)

26.53 / 30.33dB

exp56 최종 레시피 위 실측

07/18 구버전 설정에서 처음 발견된 패턴(+4.12/+7.95dB)이 exp56 최종 레시피에서도 재현됨(2026-07-27 재검증). 26k iteration은 시퀀스 길이와 무관한 고정 배치 작업이라 현재 구조로는 "실시간"이 아니라 "끝나고 한 번에 몰아서" — 다음 슬라이드는 이걸 실시간 호환으로 만들 방법에 대한 제안.

후처리를 실시간 호환으로 만들려면

- 핵심 문제: 26,000 iteration이 시퀀스 종료 후 한 번에 블로킹으로 몰림 — 온라인 시스템엔 "종료 시점"이 없음

A · 상시 백그라운드화

26k를 세션 끝에 몰아서 돌리지 않고, 이미 지나간(프론티어 밖) keyframe에 대해 GPU 유휴 사이클마다 소량씩 지속 refinement — `n_global_views(Phase7)`가 이미 하던 걸 전용 백그라운드 루프로 확장

B · 우선순위 스케줄링

`_gs_queue`가 비어 mapper가 노는 시점(exp55 부록에서 이미 발견한 유휴 구간)을 감지해 그 틈에만 refinement 스텝을 끼워 넣음 — 신규 keyframe 처리를 항상 최우선으로 선점

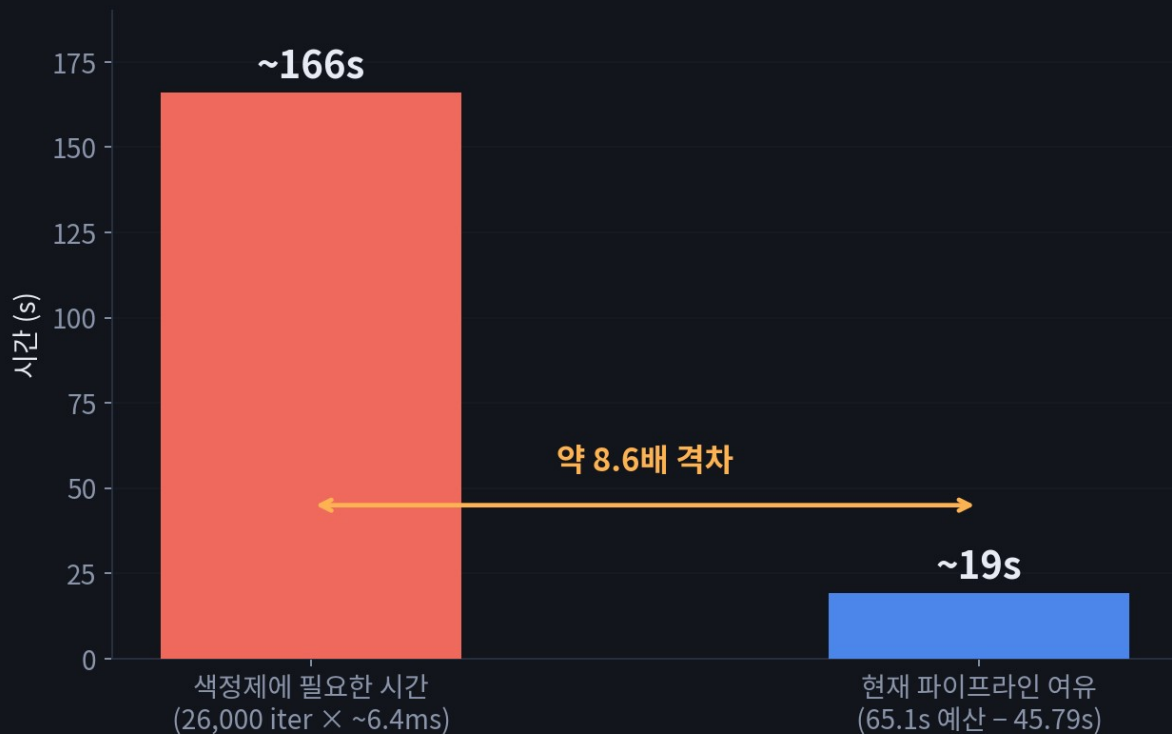
C · 반복수 자체 축소

`iters=10→7`이 온라인에서 그랬듯, 26,000도 고정 상수일 뿐 — 스윙(예: 5000/10000)해서 수확체감 지점을 찾으면 고정비용 자체를 줄일 수 있음 (저위험, 미검증)

권장: C(반복수 스윙)로 저위험 저비용 검증 먼저 → A/B(상시 백그라운드화)로 구조를 바꾸는 건 exp56 Phase7의 아이디어를 그대로 확장하는 것이라 다음 사이클 후보로 유력.

post-processed급 online — 쉽지 않은 이유

GPU 유희시간을 100% 뺀어도 부족 — 계산량 자체의 격차

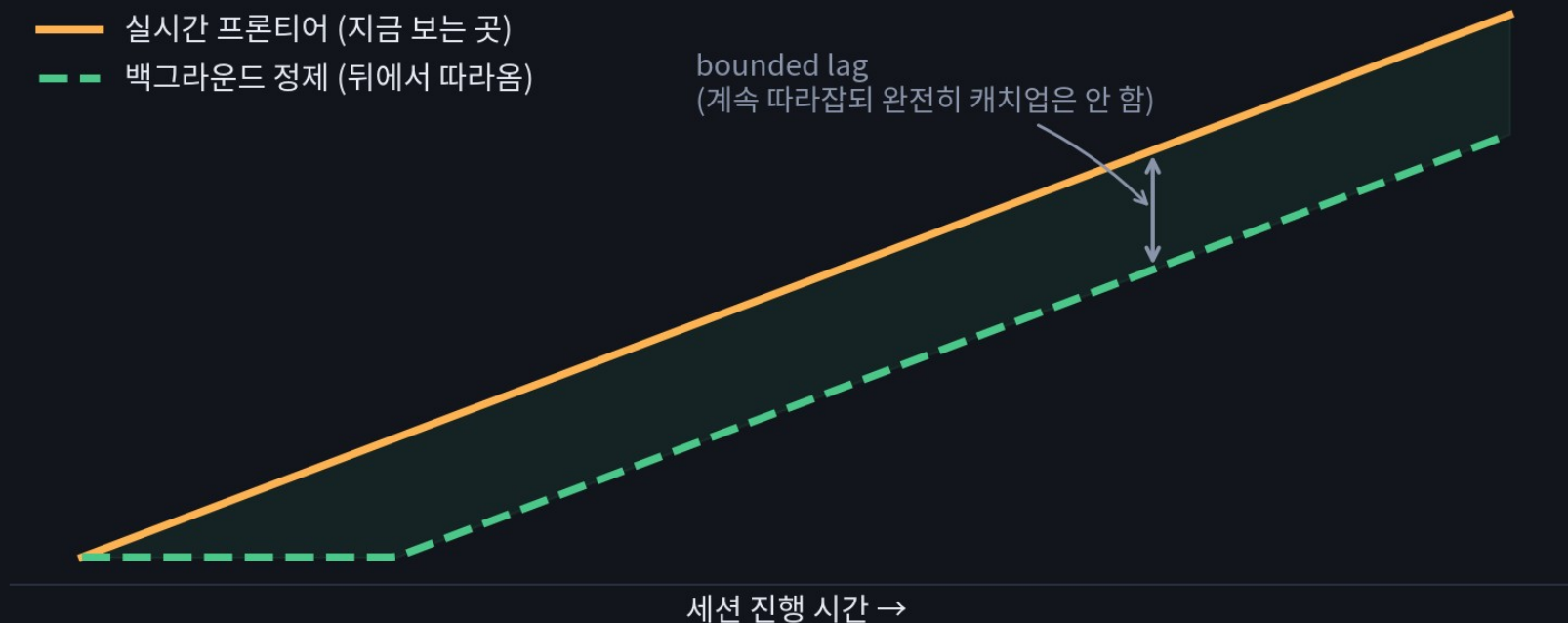


- 계산량 격차: 26,000 iter × ~6.4ms/view-op(Phase8 실측치 그대로 적용) ≈ 166~196초 필요 — 지금 파이프라인 여유는 65.1s 예산 대비 19.3초뿐, 약 8~10배 부족
- 더 근본적 문제: color_refinement는 "전체 궤적을 이미 다 본 상태"에서 모든 keyframe을 반복 방문하는 global 작업 — 라이브 스트림엔 애초에 "다 봤다"는 시점이 없음
- 즉 "매 순간 완전히 캐치업된 post-processed급"은 계산량 문제 이전에 정의상 불가능

그래서 목표를 "항상 최종 품질"이 아니라 "프론티어 + 뒤에서 점점 따라잡는 품질"로 재정의하는 게 현실적 — 다음 슬라이드가 그 그림.

현실적 목표: bounded-lag 온라인 정제

목표 재정의: "항상 최종 품질"이 아니라 "프론티어 + 뒤따라오는 정제"



루프클로저 global BA를 백그라운드로 돌리는 성숙한 SLAM 시스템들과 같은 패턴 — 프론티어(방금 본 곳)는 항상 실시간으로 그려지고, 그 뒤를 정제(refinement)가 시간차를 두고 계속 따라오며 품질을 끌어올린다. "실시간 = 항상 완성본"이 아니라 "실시간 = 프론티어가 안 밀린다 + 지나간 곳은 계속 좋아진다"로 성공 기준 자체를 바꾸는 게 이번 발견의 결론.

§ 4 · CONCLUSION

결론 한 줄

EXP53+54

실시간 최초 돌파 (0.94배)

EXP55

품질 레버 확보 (gaussian -35.9%, floater -7.5%, 비용 없음)

EXP56

"왜 안 빨라지나"를 끝까지 규명해 4단계 연속 개선 + 실패한 시도(batch화)까지 안전하게 검증

실시간을 달성하고, 그 위에서 시간을 추가로 1/4 더 줄이면서 품질까지 함께 올렸다

다음 미팅까지 목표(localization 통합 vs 커널 batch화 여부)에 대한 팀 의견 요청.